FULL-TEXT SEARCH ENGINE ON A SUFFIX-ARRAY INDEX

P NEEHARIKA _ 2520090233
G SHIVANI _ 2520090046

Abstract

- This project presents a full-text search engine based on a Suffix Array index.
- The system allows users to search large text collections using substring, prefix, and phrase queries.
- A suffix array stores the starting positions of all suffixes of a text in sorted order.
- Binary search is used to quickly locate matching portions of the text.
- Kasai's Algorithm is used to construct the Longest Common Prefix (LCP) array.
- LCP information is used to reduce unnecessary character comparisons during searching.
- The system also provides ranked search results and evaluates search performance.

Introduction

What is a Search Engine?

- A search engine finds relevant information from a collection of data based on a user's query.
- Traditional text searching can become slow when the amount of text increases.
- Efficient indexing structures are required for faster searching.

Our Approach

- We use a Suffix Array to index the text.
- The suffixes of the text are sorted lexicographically.
- Searching is performed using binary search instead of checking every position sequentially.
- The LCP array further improves the searching process.

Key Features

- Substring search
- Prefix search
- Phrase search
- Ranked results
- Efficient indexed searching

Literature Survey

Technique	Main Idea	Limitation
Naive Search	Checks the pattern at every position	Slow for large texts
Inverted Index	Maps words to their document positions	Mainly word-based
Suffix Tree	Stores all suffixes in a tree structure	High memory usage
Suffix Array	Stores sorted suffix positions	More compact than suffix trees
LCP Array	Stores common-prefix lengths	Helps reduce comparisons

Problem Statement and Objectives

Problem Statement

To design and implement an efficient full-text search engine that can search text collections using a suffix-array-based index while reducing unnecessary character comparisons.

Objectives

1. Construct a Suffix Array for the given text.
2. Construct the LCP Array using Kasai's Algorithm.
3. Implement binary-search-based pattern matching.
4. Support substring, prefix, and phrase queries.
5. Provide ranked search results.
6. Measure search performance and index size.
7. Compare the approach with a traditional inverted index.

Hardware and Software Requirements

Hardware Requirements

- Processor: Intel Core i3 or equivalent
- RAM: 4 GB or above
- Storage: 500 MB or above
- Standard keyboard and display

Software Requirements

- Operating System: Windows / Linux
- Programming Language: Java
- IDE: IntelliJ IDEA / Eclipse / VS Code

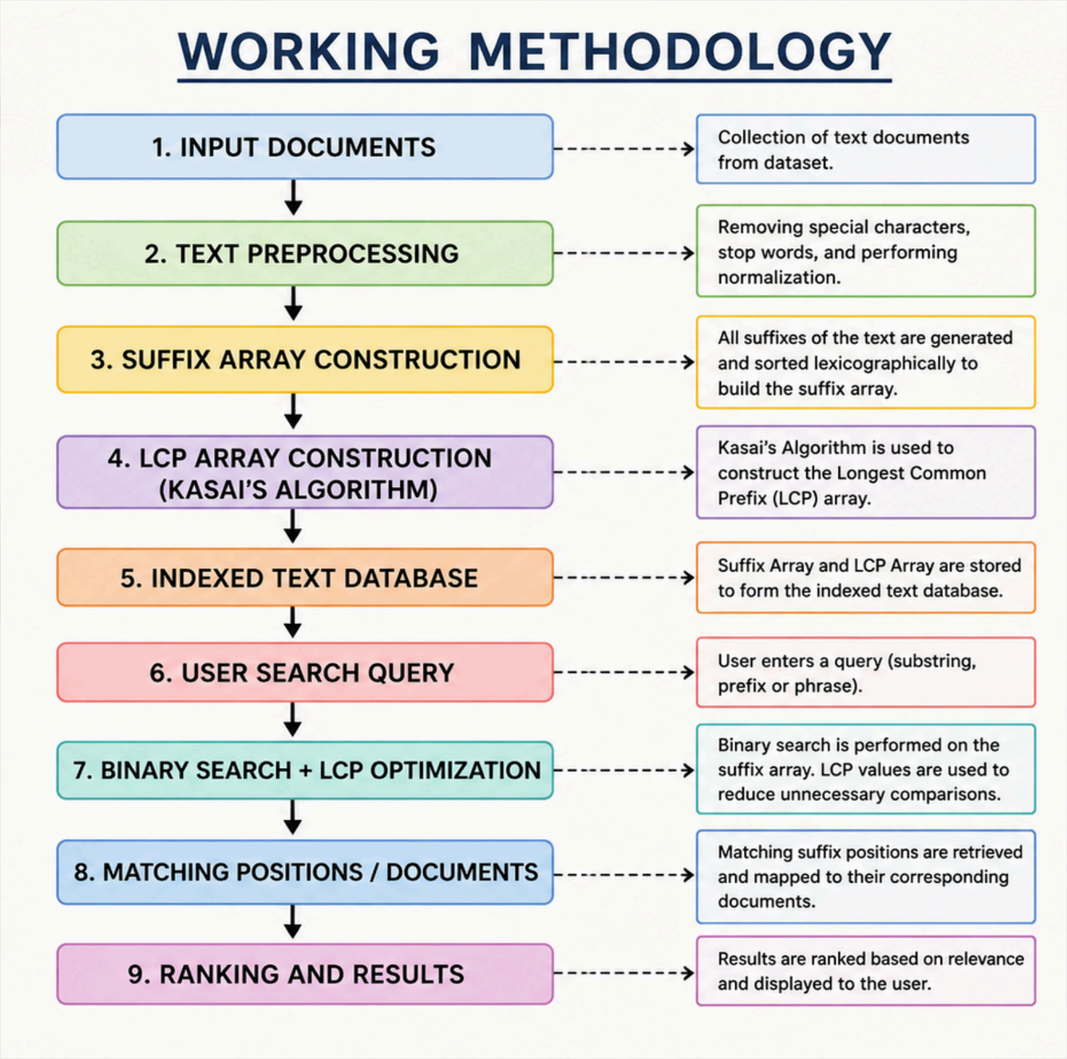
Data Structures:

- Suffix Array
- LCP Array
- Inverted Index

Algorithms:

- Binary Search
- Kasai's Algorithm
- Suffix Array Construction

System / Working Methodology



Conclusion

- A full-text search engine was implemented using a Suffix Array index.
- The suffix array provides an efficient way to locate matching patterns in text.
- Kasai‘s Algorithm efficiently constructs the LCP array.
- LCP-accelerated binary search reduces repeated character comparisons.
- The system supports different types of text queries such as substring, prefix, and phrase searches.
- The project demonstrates how advanced data structures can be applied to real-world search problems.

Future Scope

- Support larger document collections.
- Improve ranking using advanced relevance algorithms.
- Add fuzzy search for spelling variations.
- Implement parallel index construction for large datasets.
- Improve memory optimization for very large indexes.
- Develop a web-based search interface.
- Explore advanced indexing techniques and compare them with suffix arrays.
- Optimize the search engine for real-time queries.

Thank You!